ClarityAlliance

Stacks Pyth Oracle Update Security Review

Conducted by:
Kristian Apostolov, Silverologist

July 27th, 2026

Contents

1. About Clarity Alliance 2
2. Disclaimer 3
3. Introduction 3
4. About Stacks Labs 4
5. Risk Classification 4
   5.1. Impact 5
   5.2. Likelihood 5
   5.3. Action required for severity levels 5
6. Security Assessment Summary 6
7. Executive Summary 7
8. Findings 9
   8.1. Medium Findings 9
      [M-01] Valid Feeds Can Be Silently Dropped 9
   8.2. Low Findings 10
      [L-01] Invalid Channel Values Are Accepted 10
      [L-02] Duplicate Feed IDs Are Accepted and Returned 11
      [L-03] Protocol Can Be Permanently Paused if the Pause Authority Is Lost or Compromised 12
      [L-04] Trusted Signer Limit Is Lower Than EVM Reference 13
      [L-05] 16-Feed Maximum Rejects Larger Valid Pyth Pro Payloads 14
      [L-06] Duplicate Feed Properties Are Accepted With Last-Value-Wins Semantics 15
      [L-07] Fee Recipient Cannot Use the Oracle 16
   8.3. QA Findings 17
      [QA-01] High-S Signatures Are Accepted 17
      [QA-02] Buffer Size Limits Are Unnecessarily Large 18
      [QA-03] Trusted Signer Lookup Uses Linear Scan 19

1. About Clarity Alliance

Clarity Alliance is a team of expert whitehat hackers specialising in securing protocols on Stacks.

They have disclosed vulnerabilities that have saved millions in live TVL and conducted thorough reviews for some of the largest projects across the Stacks ecosystem.

Learn more about Clarity Alliance at clarityalliance.org.

2. Disclaimer

This report is not, and should not be considered, an endorsement or disapproval of any project, team, product, or asset, nor does it reflect on their economics, value, business model, or legal compliance. It does not provide any warranty regarding the absolute bug-free nature or functionality of the analyzed technology.

Nothing in this report should be used to make investment or participation decisions. It does not constitute investment advice. Instead, it reflects an extensive assessment process intended to help clients improve code quality and reduce the inherent risks associated with cryptographic tokens and blockchain systems.

Blockchain technology presents ongoing and significant risk, and each company or individual remains responsible for their own due diligence and security posture. Clarity Alliance aims to reduce attack vectors and technological uncertainty but does not guarantee the security or performance of any system we review.

All assessment services are subject to dependencies and active development. Your access to and use of any services, reports, or materials is at your sole risk on an as-is and as-available basis.

Cryptographic tokens are emergent technologies with high technical uncertainty, and assessment results may include false positives, false negatives, or other unpredictable outcomes. Smart contracts may depend on multiple external parties, remain vulnerable to internal or external exploitation, and may carry elevated risks if owner privileges remain active. Accordingly, Clarity Alliance does not guarantee the explicit security of any audited smart contract, regardless of the reported verdict.

3. Introduction

A time-boxed security review of the Stacks Pyth Oracle Update, where Clarity Alliance reviewed the Clarity contracts that integrate Pyth Lazer signed price feeds into Stacks and provided recommendations to improve the protocol's security, correctness, and maintainability.

4. About Stacks Labs

What is Stacks Labs?

Stacks Labs is the operational hub of the Stacks ecosystem - the leading infrastructure for making Bitcoin programmable, usable, and productive.

Bitcoin is the most important financial network in the world, but on its own, the base chain can't support the applications and functionality needed to unlock its full potential. Stacks solves this by enabling smart contracts, decentralized applications, and Bitcoin-native DeFi that settle directly to the Bitcoin blockchain.

What is the Stacks Pyth Oracle Update?

Clarity 5 contracts that bring Pyth Lazer ("Pyth Pro") signed price feeds to Stacks. Successor to stacks-pyth-bridge (Pythnet + Wormhole, deprecated) - simpler: no Wormhole, no Merkle proofs.

5. Risk Classification

Severity	**Impact: High**	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

5.1. Impact

- High - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- Medium - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- Low - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

5.2. Likelihood

- High - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- Medium - only a conditionally incentivized attack vector, but still relatively likely.
- Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

5.3. Action required for severity levels

- Critical - Must fix as soon as possible (if already deployed)
- High - Must fix (before deployment if not already deployed)
- Medium - Should fix
- Low - Could fix

6. Security Assessment Summary

Scope

Repository: `stx-labs/stacks-pyth-lazer`

This review covered the stateless Clarity reference implementation of the Stacks Pyth Lazer oracle. The off-chain relayer and any other off-chain infrastructure were explicitly out of scope.

Contracts

- `contracts/pyth-lazer-decoder-v1.clar`
- `contracts/pyth-lazer-oracle.clar`
- `contracts/pyth-lazer-traits.clar`

Initial Commit Reviewed

`5dc135cbc55750f9f1383629eefd846573c1a562`

Final Commit Reviewed

`bf945c12c698b008d97624b4c424d720454acb10`

7. Executive Summary

Over the course of the security review, Kristian Apostolov, Silverologist engaged with Stacks Labs to review Stacks Labs. In this period of time a total of **11** issues were uncovered.

Protocol Summary

Protocol Name	Stacks Labs
Repository	stx-labs/stacks-pyth-lazer
Report Date	July 27th, 2026

Findings Count

Severity	Amount
Medium	1
Low	7
QA	3
Total Findings	**11**

Summary of Findings

ID	Title	Severity	Status
[M-01]	Valid Feeds Can Be Silently Dropped	Medium	Resolved
[L-01]	Invalid Channel Values Are Accepted	Low	Acknowledged
[L-02]	Duplicate Feed IDs Are Accepted and Returned	Low	Acknowledged
[L-03]	Protocol Can Be Permanently Paused if the Pause Authority Is Lost or Compromised	Low	Acknowledged
[L-04]	Trusted Signer Limit Is Lower Than EVM Reference	Low	Resolved
[L-05]	16-Feed Maximum Rejects Larger Valid Pyth Pro Payloads	Low	Resolved
[L-06]	Duplicate Feed Properties Are Accepted With Last-Value-Wins Semantics	Low	Acknowledged
[L-07]	Fee Recipient Cannot Use the Oracle	Low	Resolved
[QA-01]	High-S Signatures Are Accepted	QA	Resolved
[QA-02]	Buffer Size Limits Are Unnecessarily Large	QA	Acknowledged
[QA-03]	Trusted Signer Lookup Uses Linear Scan	QA	Acknowledged

8. Findings

8.1. Medium Findings

[M-01] Valid Feeds Can Be Silently Dropped

Location:

- pyth-lazer-decoder-v1.clar#L225-L226

Description The Stacks decoder currently treats `price`, `exponent`, and `publisher-count` as required fields. If any of these fields are absent in a parsed feed, the decoder silently skips that feed and continues returning the remaining feeds.

This behavior differs from Pyth's documentation and reference implementations, where no feed fields are inherently required. According to the documentation, the `price` field is:

> Type: optional non-zero i64
> Availability: Only included if requested in subscription properties

As a result, valid payloads may decode successfully but return incomplete results, with feeds omitted solely because one of these fields was not included. This can mask data availability issues and lead to inconsistent behavior across chains.

Recommendation Consider treating all feed fields as optional, including `price`, `exponent`, and `publisher-count`.

8.2. Low Findings

[L-01] Invalid Channel Values Are Accepted

Location:

- pyth-lazer-decoder-v1.clar#L151

Description The Stacks Lazer decoder accepts any payload header channel value as a raw `uint` without validation.

In Pyth's EVM parser, the channel byte is cast to the `Channel` enum, which only permits values 0..4; values outside this range are rejected. By contrast, the Stacks implementation reads the same byte and returns it directly, without checking that it falls within the supported channel set.

As a result, a signed payload containing an invalid channel value may be accepted.

Recommendation Validate the decoded channel value in the Stacks decoder before returning the parsed payload. For example, after reading the channel byte, add a check equivalent to:

```
(asserts! (<= channel u4) ERR_INVALID_CHANNEL)
```

[L-02] Duplicate Feed IDs Are Accepted and Returned

Location:

- pyth-lazer-decoder-v1.clar#L257

Description The decoder does not enforce uniqueness of `feed-id` values within a single update.

Each feed is parsed independently: the decoder reads the `feed-id` and appends the parsed feed data to the returned `price-feeds` list. However, it does not verify whether the same `feed-id` has already appeared earlier in the update.

As a result, a single signed update may include multiple records for the same feed ID with different values. The decoder will accept the update and return all such records in order, which can introduce ambiguity for downstream consumers.

Recommendation Consider rejecting updates that contain duplicate feed IDs during decoding.

[L-03] Protocol Can Be Permanently Paused if the Pause Authority Is Lost or Compromised

Location:

- pyth-lazer-oracle.clar#L233

Description The `pause` and `unpause` functions are restricted to accounts with `ROLE_PAUSE`. Role management via `set-role` is restricted to `ROLE_GOVERNANCE`; however, `set-role` cannot be executed while the protocol is paused because it is gated by the `assert-active` check.

As a result, if the current pause authority pauses the protocol and then loses access to its key, governance cannot assign `ROLE_PAUSE` to a replacement account. The protocol would remain paused indefinitely.

Similarly, if the pause authority is compromised, an attacker can pause the protocol and prevent recovery by front-running governance attempts to replace the compromised key. Even if governance unpauses the protocol, the attacker can submit another `pause` transaction before the governance `set-role` transaction is executed.

Therefore, an unavailable or malicious pause authority can cause the protocol to enter an unrecoverable paused state.

Recommendation Enable governance recovery from a lost or compromised pause authority by removing the `assert-active` check from `set-role`, allowing role management while the protocol is paused.

[L-04] Trusted Signer Limit Is Lower Than EVM Reference

Location:

- pyth-lazer-oracle.clar#L67

Description The Stacks implementation limits the trusted signer set to 16 entries, whereas the Pyth EVM implementation supports up to 100 trusted signers.

On Stacks, trusted signers are stored as a bounded list:

```
(define-data-var trusted-signers (list 16
    {
      pubkey: (buff 33),
      expires-at: uint,
    }
  ) (list))
```

As a result, governance cannot configure more than 16 trusted signers. The decoder also iterates over this list during signer verification.

In the EVM implementation, trusted signers are stored in a fixed-size array of 100 entries:

```
TrustedSignerInfo[100] internal trustedSigners;
```

When adding a new signer, the EVM contract searches for an empty slot and only reverts once all 100 slots are filled.

Consequently, a signer configuration that is valid in the Pyth EVM implementation may not be representable on Stacks. If Pyth uses, or migrates toward, more than 16 active trusted signers, the Stacks oracle would be unable to trust the full signer set, potentially causing otherwise valid updates to be rejected.

Recommendation Consider increasing the Stacks trusted signer capacity to match the EVM reference limit of 100.

[L-05] 16-Feed Maximum Rejects Larger Valid Pyth Pro Payloads

Location:

- pyth-lazer-decoder-v1.clar#L59

Description The Stacks implementation hard-caps decoded payloads at 16 feeds.

However, the Lazer payload format encodes the feed count as a `uint8`, and the reference parsers use dynamic arrays/vectors that support parsing up to the format-level maximum of 255 feeds.

As a result, the Stacks parser rejects otherwise valid Pyth Lazer payloads that contain 17 or more feeds.

This creates a compatibility issue with broader Pyth cross-chain implementations and with larger, valid Pyth Pro payloads.

Recommendation Consider aligning the feed limit with the payload format and reference implementations by supporting up to the `uint8` maximum of 255 feeds.

[L-06] Duplicate Feed Properties Are Accepted With Last-Value-Wins Semantics

Location:

- pyth-lazer-decoder-v1.clar

Description The decoder accepts feed entries that include the same property ID more than once. During parsing, properties are processed sequentially and written to the decoded feed result as they are encountered. If a property appears multiple times, the later occurrence overwrites the earlier one.

For example, a feed can include two `price` properties:

```
price = 100
price = 200
```

The payload is accepted, and the decoded feed returns `price = 200`.

This behavior applies to all properties. The parser does not track which property IDs have already been seen within a given feed, so duplicate properties are not treated as malformed input.

As a result, payload semantics become ambiguous.

Recommendation Consider rejecting duplicate properties within each feed. The decoder can track property IDs encountered while parsing a feed and return an error if the same property ID appears more than once. This removes ambiguity and prevents consumers from relying on implicit last-value-wins behavior.

[L-07] Fee Recipient Cannot Use the Oracle

Location:

- pyth-lazer-oracle.clar#L330

Description `verify-price-feeds` charges the configured fee by calling `stx-transfer?` to transfer STX from `tx-sender` to `fee-recipient`.

If governance sets `fee-recipient` to a principal that later calls `verify-price-feeds`, and the configured fee is greater than zero, the function will attempt to transfer STX from an account to itself.

In Clarity, `stx-transfer?` rejects self-transfers, causing the entire verification call to revert even when the submitted payload is otherwise valid.

Recommendation If this scenario is expected, bypass the fee transfer when `tx-sender == fee-recipient`.

8.3. QA Findings

[QA-01] High-S Signatures Are Accepted

Location:

- pyth-lazer-decoder-v1.clar#L96

Description The decoder accepts non-canonical high-s secp256k1 signatures.

In secp256k1, any valid signature may have a malleable counterpart where `s' = n - s`, with `n` representing the curve order. If the canonical low-s form is not enforced, both signatures can recover the same public key for the same payload hash.

While this behavior does not directly impact the decoder or oracle, it may introduce risks for integrators that do not account for signature malleability. Specifically, it can break assumptions in systems that perform deduplication, replay protection, caching, or indexing based on raw update bytes or signature bytes.

Recommendation Consider rejecting non-canonical signatures by validating that `s` is in the low-s range before calling `secp256k1-recover?`.

[QA-02] Buffer Size Limits Are Unnecessarily Large

Location:

- pyth-lazer-oracle.clar#L303
- pyth-lazer-decoder-v1.clar#L83
- pyth-lazer-decoder-v1.clar#L117
- pyth-lazer-decoder-v1.clar#L138

Description
The decoder accepts `update` and `payload` buffers up to 8192 bytes; however, the current payload format cannot produce a valid update anywhere near that size.

Under the currently used EVM envelope format, the maximum possible `update` length is:

```
max-properties-length
   = 13 * property-id-byte + 4 * exists-byte + 10 * 64-bit-values + 3 * 16-bit-values
                       = 13 * 1 + 4 * 1 + 10 * 8 + 3 * 2 = 103 bytes
max-feed-length = feed-header-length + max-properties-length
                = 5 + 103 = 108 bytes
max-payload-length = payload-header + 16 * max-feed-length
                   = 14 + 16 * 108 = 1742 bytes
max-update-length = update-header + max-payload-length
                  = 71 + 1742 = 1813 bytes
```

As a result, the 8192-byte buffer limit is more than 4× larger than the largest valid update supported by the parser. Any input larger than 1813 bytes cannot be valid given the current feed/property caps and will ultimately be rejected.

Recommendation
Consider reducing the maximum accepted `update` and `payload` sizes to the largest valid size supported by the current format.

[QA-03] Trusted Signer Lookup Uses Linear Scan

Location:

- pyth-lazer-decoder-v1.clar#L493

Description Trusted signers are stored in a bounded list and validated by scanning the list during verification.

During decoding, the contract retrieves the trusted signer list and folds over it to locate the recovered public key. As a result, signer verification is linear in the number of configured signers. While the current limit is small, each verification still incurs unnecessary iteration.

Additionally, a list-based structure permits duplicate signer entries unless governance prevents them off-chain.

Recommendation Consider using a map keyed by the compressed public key as the canonical trusted signer store:

```
trusted-signers[pubkey] -> expires-at
```

This enables direct lookup of the recovered signer, eliminates linear scans, and inherently prevents duplicate signer entries.

If enumeration is still required, maintain a separate list for off-chain consumption.